SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CSA4708 – DEEP LEARNING

ASSIGNMENT 2

CO2 – DEEP FEED-FORWARD AND CONVOLUTIONAL NETWORK DESIGN

Name: Nagamokshith Koduru **Reg No:** 192424083

Department: B.Tech AIDS **Slot:** B

TASK 1 – Deep Feed-Forward Network Architecture (5 Marks)

a) Network design

The network predicts crop yield using four input features: rainfall, temperature, soil moisture and fertilizer quantity.

Layer	Number of Neurons
Input Layer	4
First Hidden Layer	8
Second Hidden Layer	4
Output Layer	1

Network architecture: Input Layer (4) → Hidden Layer 1 (8) → Hidden Layer 2 (4) → Output Layer (1).

b) Weight matrices and bias vectors

Parameter	Dimensions
W1	4 × 8
b1	1 × 8
W2	8 × 4
b2	1 × 4
W3	4 × 1
b3	1 × 1

c) Forward flow

1. The four input features are given to the input layer.
2. The first hidden layer processes the inputs using weights and biases.
3. An activation function introduces non-linearity.
4. The second hidden layer further processes the information.
5. The output layer produces the final crop-yield prediction.

TASK 2 – Forward Propagation and Activation Functions (2 Marks)

Given: Inputs = 0.6, 0.8, 0.5; weights = 0.4, 0.3, 0.2; bias = 0.1.

a) Weighted sum

$Z = (X1 \times W1) + (X2 \times W2) + (X3 \times W3) + b$

$Z = (0.6 \times 0.4) + (0.8 \times 0.3) + (0.5 \times 0.2) + 0.1$

$Z = 0.24 + 0.24 + 0.10 + 0.10 = 0.68$

Answer: $Z = 0.68$.

b) ReLU activation

$f(x) = \max(0, x)$; therefore, $f(0.68) = \max(0, 0.68) = 0.68$.

Interpretation: Since the weighted sum is positive, ReLU returns 0.68. ReLU sets negative values to zero and passes positive values through.

TASK 3 – Gradient-Based Learning and Loss Calculation (6 Marks)

Given: Predicted yield ($\hat{y}$) = 42 units; actual yield (y) = 50 units.

a) Need for a loss function

A loss function measures the difference between the actual output and the predicted output. Mean Squared Error (MSE) is suitable for crop-yield regression because it measures squared prediction errors.

b) Squared error calculation

$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = 64$.

Answer: Squared error loss = 64.

c) Interpretation

The prediction is 8 units below the actual yield. The squared error is 64; a lower loss generally indicates a closer prediction.

d) Gradient-based learning

6. The network produces a prediction.
7. The loss function calculates the prediction error.
8. Backpropagation calculates gradients of weights and biases.
9. The optimizer updates the parameters using the gradients.
10. The process repeats to reduce the loss.

TASK 4 – Gradient Descent and Weight Update (4 Marks)

Given: Initial weight = 0.5; gradient = 0.8; learning rate = 0.1.

a) Updated weight

$W(new) = W(old) - \alpha \times Gradient$

$W(new) = 0.5 - (0.1 \times 0.8) = 0.5 - 0.08 = 0.42$.

Answer: Updated weight = 0.42.

b) Opposite direction

The gradient points in the direction of steepest increase in loss. Gradient descent moves in the opposite direction to reduce the loss.

c) Learning-rate effects

- A very small learning rate makes training slow.
- A very large learning rate may overshoot the minimum and cause unstable training.
- A suitable learning rate supports stable and efficient convergence.

TASK 5 – Back-Propagation Through Multiple Layers (3 Marks)

a) Backpropagation and weight contribution

Backpropagation starts at the output layer and moves backward through the hidden layers. It calculates gradients that indicate how each weight contributes to the prediction error.

b) Role of the chain rule

The chain rule calculates how changes in parameters at earlier layers affect the final loss by combining derivatives through successive layers.

c) Weight updates

The calculated gradients are used by an optimizer, such as gradient descent, to update weights: W(new) = W(old) − learning rate × gradient.

TASK 6 – Activation Functions and Vanishing/Exploding Gradients (3 Marks)

a) Activation functions and vanishing gradients

Some activation functions, such as sigmoid and tanh, can have very small derivatives in saturated regions. Across many layers, these derivatives can make gradients extremely small, slowing learning in early layers.

b) Comparison

Function	Output Range	Advantages	Limitations
Sigmoid	0 to 1	Useful for probability outputs	Can cause vanishing gradients
Tanh	−1 to +1	Zero-centred output	Can cause vanishing gradients
ReLU	0 to infinity	Simple and often trains efficiently	Can suffer from dying ReLU

c) Effect on training

Activation-function choice affects gradient flow and learning speed. ReLU is commonly used in hidden layers to reduce the vanishing-gradient problem. Sigmoid is often used for binary-classification outputs, while Softmax is commonly used for multiclass outputs.

TASK 7 – Back-Propagation and Optimization Strategy (4 Marks)

a) Learning-rate effect

The learning rate controls the size of each parameter update. A small value may slow convergence, while a large value may cause unstable updates.

b) One training iteration

11. Forward propagation generates the predicted output.
12. The loss function calculates the error.
13. Backpropagation calculates gradients.
14. Gradient descent or another optimizer updates the weights.
15. The process repeats for the next batch or iteration.

c) Improving convergence

When loss decreases slowly: slightly increase the learning rate after checking stability, use an adaptive optimizer such as Adam, normalize inputs and choose suitable activations.

When loss fluctuates: reduce the learning rate, use mini-batches or momentum, and apply a learning-rate schedule.

d) Monitoring the loss curve

The loss curve helps identify slow convergence, unstable training and possible overfitting. Training and validation loss should both be monitored.

TASK 8 – Regularization and Overfitting Control (4 Marks)

Given: Training accuracy = 99%; validation accuracy = 78%.

a) Learning problem

The likely problem is overfitting. The large gap between training and validation accuracy suggests that the model fits the training data much better than unseen data.

b) L1 and L2 regularization

L1 regularization adds the sum of absolute weights to the loss. It can drive some weights to zero, producing a sparse model.

L2 regularization adds the sum of squared weights to the loss. It discourages excessively large weights and helps control model complexity.

c) L2-regularized loss

$L(total) = L(original) + \lambda \Sigma W^2$

Here, λ is the regularization parameter controlling the strength of the penalty, and W represents the model weights.

d) Dropout

Dropout randomly disables selected neurons during training. This reduces excessive dependence between neurons and can improve generalization to unseen data.

TASK 9 – Convolution Operation and Feature Extraction (3 Marks)

a) Suitability of convolutional layers

Convolutional layers are suitable for images because filters detect local patterns such as edges, spots, shapes and textures. These features are useful for identifying crop-leaf diseases.

b) Example convolution

Image matrix:

1	**2**	**3**
4	5	6
7	8	9

Filter:

1	**0**
0	−1

Using the top-left 2 × 2 region:

Output = (1 × 1) + (2 × 0) + (4 × 0) + (5 × −1) = 1 + 0 + 0 − 5 = −4.

Interpretation: The filter produces −4 for this region. Convolution filters respond to local pixel patterns and can help detect visual features.

c) Filters, stride and padding

- Filters: Small matrices that detect features such as edges and textures.
- Stride: The number of pixels the filter moves at each step.
- Padding: Extra pixels added around an image to control output dimensions and preserve border information.

TASK 10 – CNN Architecture and Pooling (4 Marks)

a) Function of each component

16. Input image: supplies the crop-leaf image.
17. First convolution: extracts local image features.
18. ReLU: introduces non-linearity by setting negative values to zero.
19. First pooling: reduces feature-map dimensions.
20. Second convolution: learns more complex features.
21. Second ReLU: applies non-linearity.
22. Second pooling: further reduces spatial dimensions.
23. Fully connected layer: combines extracted features for classification.
24. Output layer: predicts the class, such as healthy or diseased.

b) Max-pooling example

Input 4 × 4 matrix:

1	**3**	**2**	**4**
5	6	1	2
7	8	3	4
2	1	5	6

Using 2 × 2 max pooling with stride 2, the maximum values from the four regions are 6, 4, 8 and 6.

Output 2 × 2 matrix:

6	**4**
8	6

Thus, the feature map is reduced from 4 × 4 to 2 × 2.

c) Filter size, stride and number of filters

- Filter size determines the size of the local region examined.
- Larger stride generally reduces the spatial output dimensions.
- More filters produce more feature maps and allow the network to learn more types of features.

d) Hierarchical feature learning

Early convolutional layers learn simple edges and patterns. Middle layers learn textures, shapes and spots. Deeper layers combine these into more complex structures and disease-related features.

TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models (4 Marks)

a) Model A for numerical data

A feed-forward network is suitable for structured numerical features such as rainfall, temperature, soil moisture and fertilizer quantity. It learns relationships among these values to predict crop yield.

b) Model B for images

A CNN is suitable for leaf images because convolutional filters automatically learn visual features such as edges, spots and textures.

c) Comparison

Feature	Model A – Feed-Forward	Model B – CNN
Input	Structured numerical data	Image data
Feature extraction	Learns numerical relationships	Learns local visual features
Parameters	Dense weight connections	Shared convolution filters and dense weights
Computation	Often lower for small tabular inputs	Often higher for image processing

d) Recommendation

Use Model A for crop-yield prediction because the inputs are numerical. Use Model B for crop-disease classification because CNNs preserve spatial structure and extract image features effectively.

TASK 12 – Design Challenge: Optimize and Redesign the System (8 Marks)

A. Impact Analysis (2 Marks)

a) Impact of larger data and images

The numerical dataset increases from 10,000 to 500,000 crop records, and the system must also process 100,000 leaf images. This increases storage, memory, computation and training-time requirements. Image data also requires an architecture capable of learning spatial features.

b) Limitations of feed-forward networks for images

- Images are commonly flattened into one-dimensional vectors, losing explicit spatial arrangement.
- Dense connections can create a very large number of parameters.
- The network does not naturally exploit local image patterns and shared filters.
- Training can become computationally expensive.

CNNs are generally more suitable for image data.

B. Evaluation of the Existing Design (2 Marks)

a) Existing model performance

High training accuracy and lower validation performance indicate overfitting and weak generalization. The model should be evaluated using both training and validation metrics.

b) Increasing depth and neuron count

Simply adding layers and neurons is not necessarily the best solution. It can increase computation, training time and overfitting risk, and may make optimization harder. Architecture should be selected according to the data and task.

C. Adaptation or Redesign (2 Marks)

a) Proposed architectures

Model 1 – Feed-forward network for numerical crop data:

Input (4 features) → Dense (64) → ReLU → Dense (32) → ReLU → Output (1, predicted crop yield).

Model 2 – CNN for leaf images:

Input Image → Conv2D (32 filters) → ReLU → Max Pooling → Conv2D (64 filters) → ReLU → Max Pooling → Flatten → Dense → Output (healthy/diseased).

These models should be trained separately using the corresponding numerical and image datasets.

b) Optimization and regularization

Method	Purpose
Adam	Adaptive learning-rate optimization; a practical initial choice.
SGD	Gradient-based parameter updates.
Learning-rate adjustment	Improves convergence and training stability.
Dropout	Reduces overfitting by randomly disabling units during training.
L2 regularization	Penalizes large weights.
Batch normalization	Normalizes intermediate activations and can stabilize training.
ReLU	Common hidden-layer activation that supports gradient flow.

D. Evidence-Based Defence (2 Marks)

a) Experiment comparing original and redesigned systems

Train and evaluate the original and redesigned approaches using the same data splits and comparable conditions. Record training loss, validation loss, validation performance and training time.

Metric	Purpose
MSE	Measures squared error for crop-yield regression.
MAE	Measures average absolute crop-yield prediction error.
Accuracy	Measures the proportion of correctly classified images.
Precision	Measures the proportion of predicted positives that are correct.
Recall	Measures the proportion of actual positives detected.
F1-score	Balances precision and recall.
Training time	Measures computational time required for training.

b) Recommendation

Redesign the system using a feed-forward network for numerical crop data and a CNN for leaf images. This matches each architecture to its data type, supports image feature extraction and allows regularization and optimization methods to address overfitting. The final choice should be confirmed by measured validation performance and training time.

Additional Design Considerations

1. Comparison of optimization algorithms

Algorithm	Description
SGD	Updates parameters using gradients; the gradient may be computed from a sample or batch.
Mini-batch SGD	Uses a small batch of samples for each update.
Momentum	Uses a moving contribution from previous gradients to smooth updates.
Adam	Combines momentum-like estimates with adaptive

	parameter-wise learning rates.
RMSProp	Adapts learning rates using a moving average of squared gradients.

2. Learning-rate schedule

A starting learning rate such as 0.001 may be used with Adam. Monitor validation loss and reduce the learning rate when progress plateaus. The schedule should be selected based on validation experiments.

3. Label-preserving image augmentation

- Small rotations.
- Horizontal flips, if leaf orientation does not change the class.
- Random crops and modest zooming.
- Small brightness or contrast changes.
- Small translations.

Augmentations should preserve the disease label and should not distort important symptoms.

4. Comparison of CNN architectures

Architecture	**Description**
LeNet	A relatively small, early CNN architecture suitable for simple image tasks.
AlexNet	A deeper CNN with multiple convolutional layers; can require more computation.
ResNet	Uses residual connections to help train deeper networks.
EfficientNet	Balances network depth, width and input resolution for computational efficiency.

Recommendation: EfficientNet is a candidate when computational resources are limited and a strong accuracy-efficiency balance is desired. ResNet is another option when resources allow. The final architecture should be selected through validation experiments using the available image size and hardware.

CONCLUSION

Deep learning supports smart agriculture through crop-yield prediction and crop-disease identification. Feed-forward networks are suitable for numerical agricultural data, while CNNs are effective for image classification. Backpropagation, gradient descent, optimization and regularization techniques help improve model learning and generalization. A redesigned system combining a feed-forward network and a CNN is a suitable approach for the larger dataset and the additional image-classification requirement.